A Comparative Analysis of Machine Learning Models for Financial Forecasting

Jose Luna
Johns Hopkins University

March 11, 2026

Abstract

We evaluate three models (ARIMA, XGBoost, and Random Forest) for forecasting next-day stock prices, and integrate these predictions into a Top-K trading strategy framework. Using ten years of daily prices from 25 large-cap technology stocks, we generate forecasts and backtest strategy performance against a benchmark. Across all models, predictive accuracy remains limited, with directional accuracy hovering near 50%. Although the best model-strategy combination slightly outperforms the benchmark during the validation and test windows, our findings suggest that these models, when trained solely on technical indicators and daily prices, do not yield meaningful predictive power for next-day returns.

Table of Contents

1 Introduction **2**

2 Candidate Models **3**
2.1 ARIMA . . . 3
2.2 XGBoost . . . 4
2.3 Random Forest Regression . . . 6

3 Model Evaluation **7**
3.1 Experiment Design . . . 7
3.2 Hyperparameter Tuning . . . 8
3.3 Results . . . 9
3.4 Discussion . . . 10

4 Top-K Trading Strategy **11**
4.1 Hyperparameter Tuning . . . 12
4.2 Hyperparameter Tuning Results . . . 13

5 Trading Experiment **14**
5.1 Results . . . 14
5.2 Discussion . . . 15

6 Conclusion and Future Work **15**

1 Introduction

Financial markets are among the most challenging domains for predictive modeling. Stock prices are driven by a complex combination of company fundamentals, investor sentiment, and the large-scale economy. Short-term movements, in particular, are notoriously difficult to forecast, due in part to a low signal-to-noise ratio and statistical properties such as non-stationarity [1].

However, decades of research have explored the possibility exploiting market inefficiencies through time-series techninques and machine learning. In particular, classical approaches such as autoregressive (AR) and ARIMA models [2], and nonlinear and nonparametric models such as Random Forests [3], Gradient Boosting Machines [4], and Recurrent Neural Networks [5] have all shown promise in this regard.

In this project, we design, implement and evaluate several models for short-term stock price prediction within the U.S technology sector. Specifically, our task is to forecast short-term returns for the top 25 technology stocks by market cap, and to translate these forecasts into daily trading decisions within a simulated portfolio. The core question is whether machine learning techniques can outperform statistical baselines such as ARIMA, and whether certain trading strategies are more profitable than others.

The project has two main goals:

1. To develop and compare three models (ARIMA, XGBoost and Random Forest Regression) to predict short-term stock returns based on historical price data.
2. To implement a portfolio management algorithm that converts model predictions into buy/sell signals and evaluates cumulative returns against a passive benchmark (the VGT ETF).

Each model starts with a historical dataset of daily adjusted close prices information obtained from Yahoo Finance. The target variable is the h-day forward return, defined as

$$r_{t+h} = \frac{P_{t+h} - P_t}{P_t},$$

where P_t is the closing price at time t. Throughout the project, h is set to one trading day.

2 Candidate Models

2.1 ARIMA

The Autoregressive Integrated Moving Average (ARIMA) model is an established approach for modeling time-dependent series. It assumes that each observation in a time series is a linear combination of past values and past forecasting error. The first step of the ARIMA model is to *difference* the time series enough times to remove any trends or seasonality. This means replacing the series x_t with the changes between consecutive observations, such as

$$y_t = x_t - x_{t-1}.$$

Having differenced the series d times (until we reach a roughly stationary time series with the mean and covariance constant over time), the ARIMA(p, d, q) model is

$$y_t = c + \sum_{i=1}^{p} \phi_i \, y_{t-i} + \sum_{j=1}^{q} \theta_j \, \varepsilon_{t-j} + \varepsilon_t,$$

where:

- y_t is the value (or return) at time t.
- ε_t is a random error term.
- ϕ_i and θ_j are autoregressor and moving-average coefficients, respectively.
- c is a constant term.

Intuitively, the model consists two parts:

- An *autoregressive* part $\sum \phi_i \, y_{t-i}$, which regresses y_t on its prior values to capture momentum or mean-reversion behavior. For example, AR(1,0,0) would be $y_t = c + \phi_1 y_{t-1} + \varepsilon_t$. A positive ϕ_1 would imply persistence of returns, while a negative ϕ_1 suggests mean reversion.
- A *moving-average* part $\sum \theta_j \, \varepsilon_{t-j}$ which lets the model learn patterns in the residual noise.

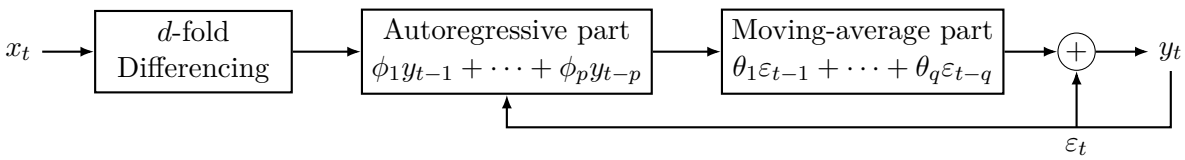

Figure 1: Block diagram of an ARIMA(p, d, q) model.

The model's parameters can be found via maximum likelihood estimation. Under the assumption that the errors ε_t are independent and normally distributed, the likelihood of the observed data can be written in closed form and optimized numerically with respect to all parameters. The orders (p, q) can be selected by comparing models with different (p, q) combinations using information criteria such as the *Akaike Information Criterion* (AIC)

$$\text{AIC} = 2k - 2\ln(\hat{L}),$$

where k is the number of parameters and $\hat{L}$ is the maximized likelihood. In this project, however, we delegate the model selection and parameter estimation to the software package `pmdarima`, which tests a range of (p, d, q) values, fits each model via maximum likelihood, and chooses the configuration that minimizes the AIC.

ARIMA is appealing as a model because it is simple and interpretable, yet can still capture complex dynamics. However, as a linear model, it may struggle with nonlinear relationships. Thus, ARIMA will serve as a baseline for our analyses, while we also consider more sophisticated machine-learning approaches.

2.2 XGBoost

Our next candidate model is Extreme Gradient Boosting (XGBoost), a gradient boosting algorithm that has been shown to capture rich, nonlinear interactions among multiple predictive features [6]. Gradient boosting algorithms combine many weak learners (typically shallow regression trees) into a single strong predictor, yielding a model of the form

$$\hat{y}_i = \sum_{m=1}^{M} f_m(x_i)$$

where each f_m is a regression tree mapping the feature vector x_i to a numerical output. The model is trained stage by stage: at step m, a new tree f_m is added to reduce the residual errors of the current ensemble. Formally, the algorithm minimizes a regularized objective

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} l\Big(y_i, \hat{y}_i^{(m-1)} + f_m(x_i)\Big) + \Omega(f_m),$$

where $l(y, \hat{y})$ is the loss function (typically squared error) and $\Omega(f_m)$ penalizes complex trees. By expanding the loss function using a second-order Taylor approximation around the previous prediction $\hat{y}_i^{(m-1)}$, we obtain

$$\mathcal{L}^{(m)} \approx \sum_i \left[g_i f_m(x_i) + \tfrac{1}{2} h_i f_m(x_i)^2 \right] + \Omega(f_m),$$

where $g_i = \partial_{\hat{y}_i} l(y_i, \hat{y}_i^{(m-1)})$ and $h_i = \partial^2_{\hat{y}_i} l(y_i, \hat{y}_i^{(m-1)})$ are the first and second derivatives of the loss with respect to the predictions. Each new tree f_m is then fit to minimize this objective.

XGBoost extends this idea with several enhancements:

1. It specifies an explicit regularization term $\Omega(f_m) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2$, where T is the number of leaves in the tree and w_j are the leaf weights. This discourages overly deep or dense trees and helps prevent overfitting.

2. It incorporates a technique known as *shrinkage* with a learning rate η, scaling each new tree's contribution as $\hat{y}^{(m)} = \hat{y}^{(m-1)} + \eta f_m(x)$.

3. It uses feature subsampling to prevent overfitting.

Together, these improvements make XGBoost faster and more robust than classical gradient boosting algorithms. For the purposes of brevity, the details of the split-finding algorithm are omitted; these can be found in [6]. In this project, we use the open-source XGBoost library to run our model.

Hyperparameters

- **learning-rate**: the "shrinkage parameter" that scales each tree's contribution.
- **max-depth**: the maximum depth of each regression tree.
- **n-estimators**: the number of boosted trees in the final ensemble.
- **subsample**: the fraction of training samples used when constructing each tree.
- **colsample-bytree**: the fraction of features subsampled for each tree

In our project, each training example corresponds to a single trading day, with several technical indicators picked from the standard suite commonly used in financial forecasting [7], [8]:

- **Lagged Returns:** The previous day's return, as well as the past 5- or 10-day returns.
- **Moving Averages (MA):** Rolling averages (5-day, 10-day and 20-day).
- **Moving Average Convergence Divergence (MACD):** The difference between 12-day and 26-day exponential moving averages (EMA). We also include a 9-day EMA of the MACD itself, known as the **MACD Signal**.
- **Relative Strength Index (RSI):** A bounded momentum oscillator (0-100) that captures overbought and oversold conditions.
- **Rolling Volatility:** The standard deviation of returns over 5-, 10- and 20-day windows.

The target variable y_i is the log-return over one day:

$$y_i = \ln\left(\frac{P_{t+1}}{P_t}\right)$$

We choose the log-return as the target because these tend to be more stationary than raw prices, reducing the impact of long-term drift and making the problem better behaved. Although powerful, XGBoost's flexibility will potentially come at the cost of interpretability and a need for careful hyperparameter tuning. Nevertheless, we expect to yield equal or superior predictive accuracy compared to ARIMA.

2.3 Random Forest Regression

Random Forest Regression (RF) is an ensemble learning method that constructs a large collection of decision trees and averages their predictions to produce a more stable, robust estimator [9]. Each tree in the forest is trained on a different bootstrap sample of the training data, and at each split the algorithm considers only a random subset of the available features. Formally, if $f_m(x)$ denotes the prediction of the m-th tree, the Random Forest predictor is

$$\hat{y}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x),$$

where M is the total number of trees. Since each tree is trained on a resampled version of the data and uses randomized feature selection, the individual trees are de-correlated. Averaging them reduces variance without substantially increasing bias, a property that makes this method particularly effective in noisy settings such as financial forecasting.

Each tree in the forest is built using CART (Classification and Regression Trees). At every split, the algorithm selects the feature and threshold that maximally decreases the the sum of squared residuals. Because the training data is randomly sampled with replacement, some observations appear multiple times in a bootstrap sample while others are omitted. In this project, we use *sklearn*'s build in *RandomForestRegressor*.

Hyperparameters

- **M**: the number of trees in the forest.
- **max-depth**: the maximum depth of each tree.
- **max-features**: the number of predictor variables the algorithm is allowed to consider when searching for the best split at each node. Settings considered are:
  - **"sqrt"**: use $\sqrt{d}$ features at each split, where d is the total number of features.
  - **"log2"**: use $\log_2(d)$ randomly chosen features at each split.
- **min-samples-split**: the minimum number of samples required to split an internal node.
- **min-samples-leaf**: the minimum number of samples in each leaf.

In our implementation, each decision tree is trained using the same feature set used by the XGBoost model: lagged returns, moving averages, MACD and MACD signal, RSI, rolling volatility measures, and lagged price levels. The target variable remains the next-day log-return. Because each tree partitions the feature space differently, the ensemble should be able to capture a wide variety of nonlinear interactions and threshold effects in the technical indicators, patterns that linear models such as ARIMA cannot easily represent.

Compared to XGBoost, Random Forests are generally simpler to tune and less sensitive to hyperparameter choices, but they lack the former's ability to focus iteratively on the most

difficult examples. Thus, we expect Random Forests to perform similarly to XGBoost, but with potentially higher variance.

3 Model Evaluation

Before considering trading strategies, we evaluate the three candidate models purely on the basis of their ability to predict stock prices 1 day in the future. Specifically, we consider the two following metrics:

- **Mean Absolute Percentage Error (MAPE)**: This is defined as:

$$\text{MAPE} = \frac{1}{T}\sum_t \frac{|A_t - F_t|}{A_t}$$

  where A_i is the actual price of the stock, F_i is the forecasted price, and T is the total number of observations.

  MAPE is appealing because it expresses forecast error in percentage terms, which makes it easy to interpret across different stocks and price scales. In addition, MAPE is more robust than squared-error metrics because it is less sensitive to large deviations.

- **Directional Accuracy (DA)**: This is defined as:

$$\text{DA} = \frac{1}{T}\sum_t \mathbf{1}_{sign(A_t - A_{t-1}) = sign(F_t - F_{t-1})}$$

  In other words, DA measures the percentage of the time the model correctly predicts whether the price goes up or down. This is a common metric for financial forecasting because a strategy may remain profitable even with substantial price-level errors as long as it correctly forecasts the sign of the movement.

3.1 Experiment Design

To ensure a fair comparison across all three models, we evaluate them on the same chronological data split. Each model is trained, validated, and tested on identical date ranges:

- **Training period:** `2015-01-01 - 2025-01-12`
- **Validation period:** `2025-01-13 - 2025-08-12`
- **Test period:** `2025-08-13 - 2025-11-13`

For each model, features are constructed using only information available up to time t, and the model produces one-step-ahead price forecasts $\hat{P}_{t+1}$ throughout the test window.

ARIMA: For each trading day in the test range, ARIMA is refit using all data from the training range up through day t. The model then produces a forecast for day $t+1$. This rolling refit procedure is standard for ARIMA models and helps preserve stationarity assumptions.

XGBoost and Random Forest: These models are trained once on the training period using he engineered technical indicators. Hyperparameters are selected and tuned on the validation

period, and the final tuned model is then used to forecast raw prices over the test period. For both models, the predicted log-return $\hat{y}_t$ is converted back to a price prediction using

$$\widehat{P}_{t+1} = P_t \cdot e^{\hat{y}_t}.$$

This design ensures that all three models make predictions for the exact same test dates, enabling direct comparison.

3.2 Hyperparameter Tuning

Only the machine-learning models (XGBoost and Random Forest) require hyperparameter tuning. ARIMA does not require manual tuning because we rely on the `auto_arima` procedure, which performs automatic model selection as outlined in the *Candidate Models* section. For XGBoost and Random Forest, we evaluate each parameter combination for each ticker on the validation period, and score the performance based on Directional Accuracy. The best-performing parameter set for that ticker is cached and reused in subsequent runs. Note that this means we end up fitting 25 independent models for each (1 for each of the 25 tickers we are considering). The hyperparameter ranges tested are as follows:

XGBoost:

- **max_depth** $\in \{4, 5, 6, 7\}$
- **learning_rate** $\in \{0.05,\ 0.75,\ 0.1, 0.125, 0.2\}$
- **n_estimators** $\in \{200, 300, 400, 500\}$
- **subsample** $\in \{0.4, 0.5, 0.6, 0.7, 0.8\}$
- **colsample_bytree** $\in \{0.4, 0.5, 0.6, 0.7, 0.8\}$

Random Forest:

- **n_estimators** $\in \{200, 300, 400, 500\}$
- **max_depth** $\in \{\text{None}, 1, 2, 3, 4, 5\}$
- **max_features** $\in \{$`"sqrt"`, `"log2"`$\}$
- **min_samples_split** $\in \{2, 3, 4, 5\}$
- **min_samples_leaf** $\in \{1, 2, 3, 4\}$

3.3 Results

The table below summarizes the performance of all three models across 25 tickers.

Ticker	**ARIMA**		**XGBoost**		**Random Forest**	
	MAPE	**DA**	**MAPE**	**DA**	**MAPE**	**DA**
AAPL	0.010	0.585	0.012	0.500	0.012	0.545
ADBE	0.012	0.385	0.017	0.364	0.014	0.409
AMD	0.027	0.492	0.035	0.364	0.031	0.439
AMZN	0.015	0.508	0.021	0.561	0.016	0.530
APP	0.030	0.523	0.046	0.485	0.048	0.485
ASML	0.017	0.477	0.019	0.455	0.019	0.470
AVGO	0.022	0.492	0.020	0.530	0.022	0.455
BABA	0.024	0.492	0.038	0.500	0.028	0.485
CRM	0.016	0.462	0.016	0.470	0.016	0.439
CSCO	0.011	0.446	0.012	0.439	0.010	0.455
GOOG	0.014	0.523	0.020	0.485	0.016	0.500
IBM	0.013	0.508	0.020	0.439	0.026	0.485
INTC	0.029	0.400	0.026	0.455	0.028	0.485
META	0.013	0.477	0.017	0.379	0.013	0.500
MSFT	0.008	0.554	0.011	0.485	0.013	0.530
MU	0.028	0.523	0.042	0.439	0.035	0.424
NFLX	0.012	0.523	0.019	0.530	0.014	0.621
NVDA	0.021	0.569	0.028	0.394	0.021	0.470
ORCL	0.035	0.492	0.037	0.439	0.044	0.455
PLTR	0.028	0.446	0.025	0.530	0.024	0.500
SAP	0.011	0.446	0.016	0.455	0.015	0.500
SHOP	0.021	0.462	0.033	0.485	0.023	0.515
TCEHY	0.012	0.508	0.019	0.485	0.014	0.439
TSLA	0.026	0.462	0.028	0.455	0.027	0.439
TSM	0.020	0.538	0.024	0.561	0.021	0.561
TOTAL	**0.019**	**0.492**	**0.024**	**0.467**	**0.022**	**0.485**

Table 1: Model comparison over the test period, showing MAPE and Directional Accuracy (DA) for ARIMA, XGBoost, and Random Forest models.

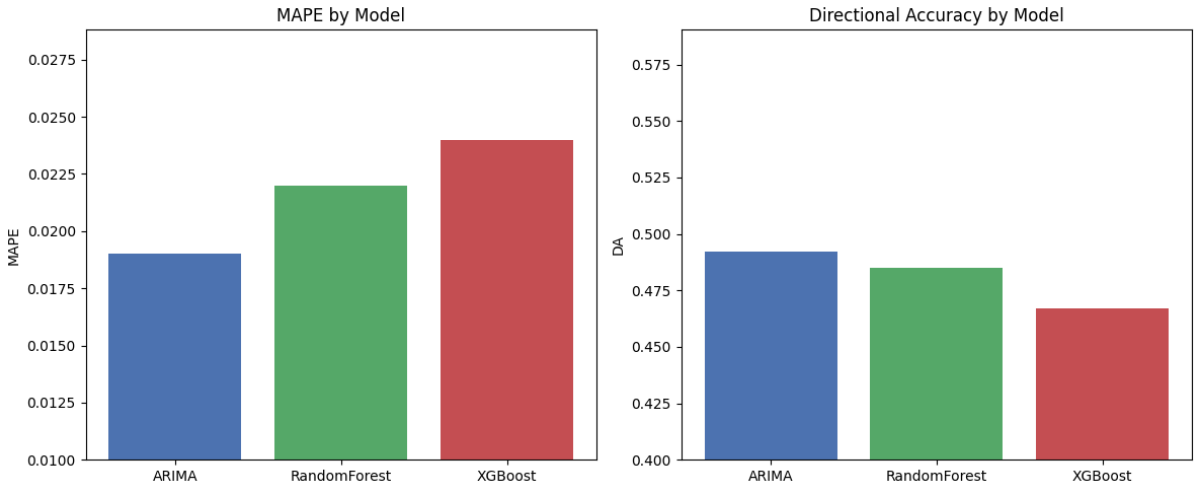

Figure 2: Total MAPE and Directional Accuracy by Model

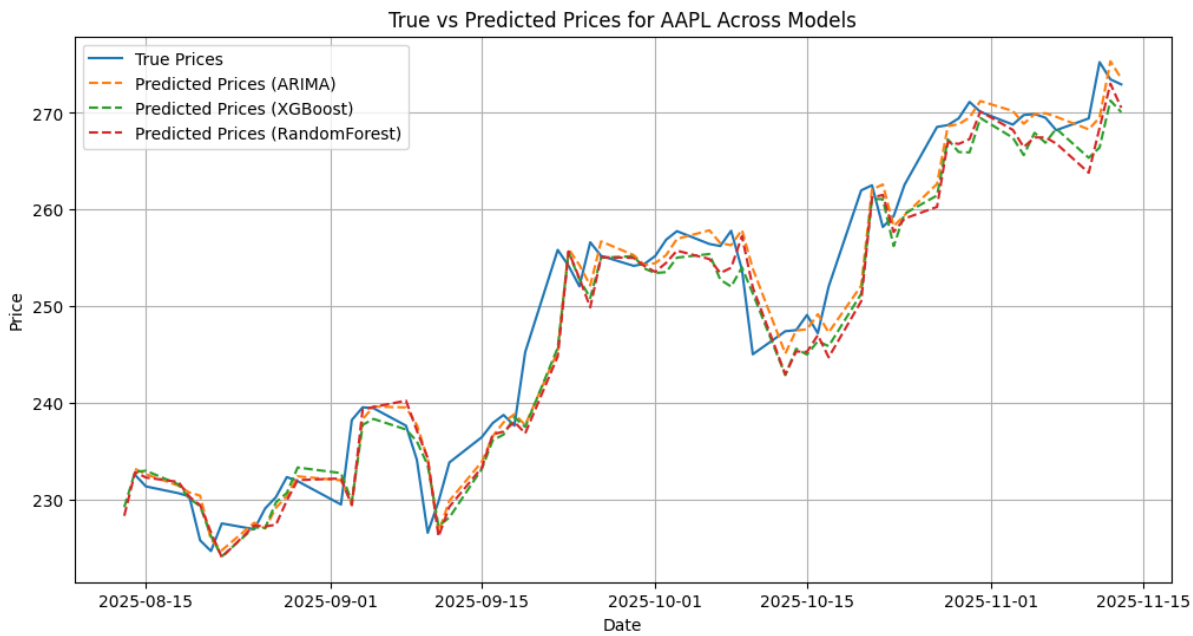

Figure 3: True vs. Predicted Prices for AAPL across Models

3.4 Discussion

All three models achieve relatively low prediction errors, with MAPE values typically between 1% and 3%. However, their directional accuracy remains close to chance (roughly $45\% - 55\%$). Although ARIMA attains slightly better performance than the machine-learning models, and Random Forest marginally outperforms XGBoost in both metrics, the overall differences are small and no model shows signs of true predictive power.

The predicted price curves (Figure 3) reveal that all models closely track the true price series, but with a one-day lag. This indicates that the models are mostly relying on recent prices and short-term trends rather than capturing deeper predictive structure. Since daily stock returns exhibit high levels of noise and near-random walk behavior, even flexible nonlinear models such

as XGBoost and Random Forest struggle to extract directional signals without incorporating additional information (e.g., volume, macroeconomic indicators, intraday features, or cross-ticker relationships).

In summary, while ARIMA shows a modest edge, none of the approaches demonstrates strong predictive power in the 1-day-ahead setting. These findings are consistent with the efficient market hypothesis, which suggests that short-term price movements contain little exploitable structure for purely statistical models. However, the slightly higher performance of ARIMA might also suggest that in the absence of richer feature sets, simple time-series structure is more informative than nonlinear transformations.

4 Top-K Trading Strategy

A trading strategy specifies how a portfolio should evolve over time based on the predictions for next-day prices. At each date, a strategy observes (i) the historical price series up to and including that day, (ii) the model's forecast for the following day, and (iii) the current portfolio state (cash and holdings). It then produces a set of buy or sell orders for the next day. The backtesting engine executes these orders using the open prices of the next trading day, updates portfolio values, and continues until the end of the test window.

Among the many possible strategies, we focus on a *Top-K Strategy* family. This strategy selects a subset of the most attractive stocks each day based on predicted next-day returns. On each trading date, we compute a score for every stock and rank them according to this score. We then choose the top K, and allocate capital accordingly.

Score Computation. For each stock on date t, the model produces a predicted next-day price $\hat{P}_{t+1}$. We convert this to a predicted return,

$$\hat{r}_{t+1} = \frac{\hat{P}_{t+1} - P_t}{P_t}$$

To adjust for risk, we also estimate volatility using a rolling window of historical returns,

$$\sigma_t = \mathrm{Std}(r_{t-w+1}, \ldots, r_t)\,,$$

where w is a configurable window length. We then form a *Sharpe-like* score,

$$S_t = \frac{\hat{r}_{t+1}}{\sigma_t + \varepsilon},$$

which mirrors the classical Sharpe ratio: expected return divided by estimated risk.

Weight Construction. Once the top K stocks are identified, we assign portfolio weights with two possible modes:

- **Equal weighting:** each selected stock receives weight $1/K$.
- **Proportional weighting:** weights are proportional to their scores.

All weights are then renormalized to sum to 1. To avoid excessive turnover, we also apply am optional `smoothing` parameter (α) to blend new target weights with those from the previous day.

Rebalancing Each day, the strategy computes the desired dollar allocation for each stock. Only a fraction of available portfolio value is used each day, controlled by the `investment-proportion` parameter (p). If `always-buy = False`, the strategy refrains from opening new positions on days where the top-K set does not change, though it will still rebalance existing positions if needed. Orders are then generated to align current holdings with the target portfolio.

Strategy Configuration Options. To allow for extensive experimentation, we add the following hyperparameters to the Top-K Strategy:

- **K**: number of top-ranked stocks selected each day.
- **investment-proportion** (p): fraction of current portfolio value allocated daily.
- **risk-mode**: whether to use predicted returns as the scoring function (`"none"`) or the risk-adjusted scores (`"sharpe"`).
- **proportional**: whether target weights are proportional to scores or equal-weighted.
- **smoothing** (α): degree of exponential smoothing applied to daily target weights.
- **vol-window** (w): rolling volatility window length.
- **always-buy**: controls whether new purchases are made only when the Top-K set changes.

4.1 Hyperparameter Tuning

To find the best parameters, we perform a grid search over a range of hyperparameters. For each configuration, we run the full backtesting pipeline on the validation window (2025-08-13 to 2025-10-12) using forecasts from all three models. Each configuration is scored based on the final portfolio value and the best-performing setting is selected. The hyperparameters explored are:

- **K** $\in \{3, 5, 7, 10\}$
- **investment-proportion** $\in \{0.5, 0.7, 0.9, 1.0\}$
- **risk-mode** $\in \{\texttt{none}, \texttt{sharpe}\}$
- **proportional** $\in \{\texttt{True}, \texttt{False}\}$
- **smoothing** $\in \{0.2, 0.5, 0.8, 1.0\}$
- **vol-window** $\in \{20, 30\}$
- **always-buy** $\in \{\texttt{True}, \texttt{False}\}$

4.2 Hyperparameter Tuning Results

Table 2 summarizes the best-performing configurations (top 3), followed by the lowest-performing ones (bottom 3). Each row lists the hyperparameters, the predictive model used, and the resulting final portfolio value.

Table 2: Top-K Strategy Hyperparameter Search Results

K	p	**risk-mode**	**prop.**	α	w	**always-buy**	**Model**	**Final Value ($)**
7	**1.0**	**sharpe**	**False**	**1.0**	**30**	**True**	**RandomForest**	**113,971.88**
7	1.0	sharpe	False	1.0	30	False	RandomForest	113,947.37
5	0.95	none	False	0.8	20	True	ARIMA	113,212.64
					...			
3	1.0	sharpe	False	0.8	30	True	XGBoost	96,100.55
3	0.95	sharpe	False	0.8	30	True	XGBoost	95,735.79
3	1.0	sharpe	False	0.9	20	False	XGBoost	95,678.10

The best performing strategy used the Random Forest model, with $K = 7$, `investment-proportion` $= 1.0$, `risk-mode` $=$ `sharpe`, `proportional` $=$ `False`, $\alpha = 1.0$, $w = 30$ and `always-buy` $=$ `True`. We will use this strategy for the final trading experiment. Below, we plot the portfolio value from this strategy over time, along with the worst strategy tested and the VGT ETF as a benchmark.

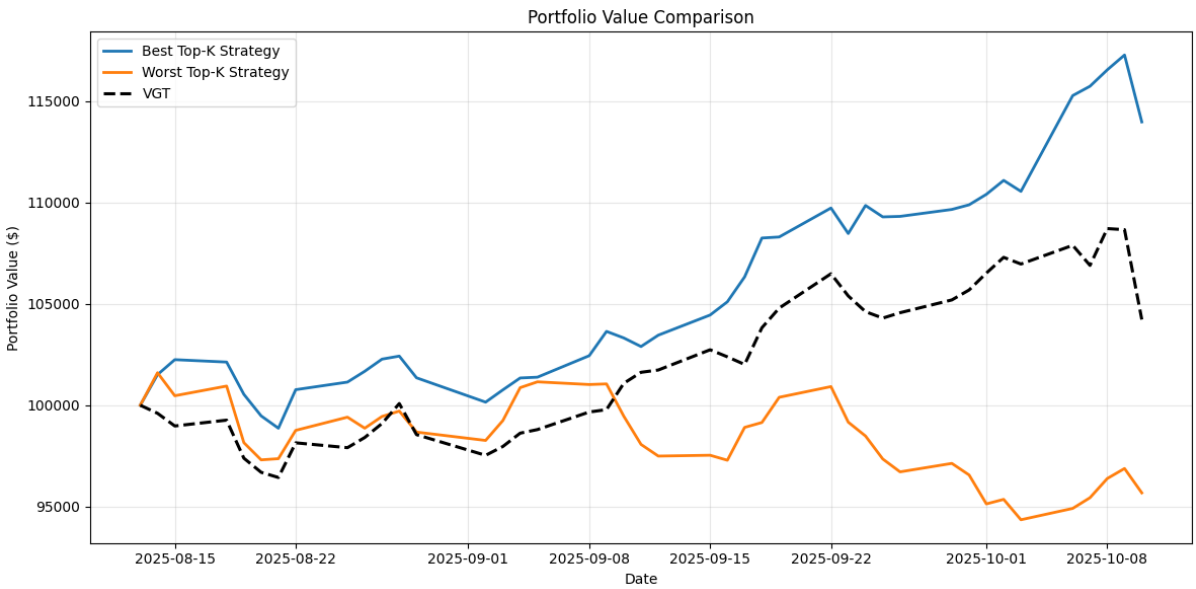

Figure 4: Comparison of portfolio value over time for the best strategy found, the worst strategy found, and the VGT ETF.

5 Trading Experiment

Having selected the best Top-K strategy during the validation phase, we now evaluate its performance over a month-long test window from 2025-10-13 to 2025-11-13, starting with an initial portfolio of $100,000 in cash and no holdings. We compare the resulting portfolio trajectory against a benchmark ETF (VGT), scaled so that both series begin at the same initial value.

5.1 Results

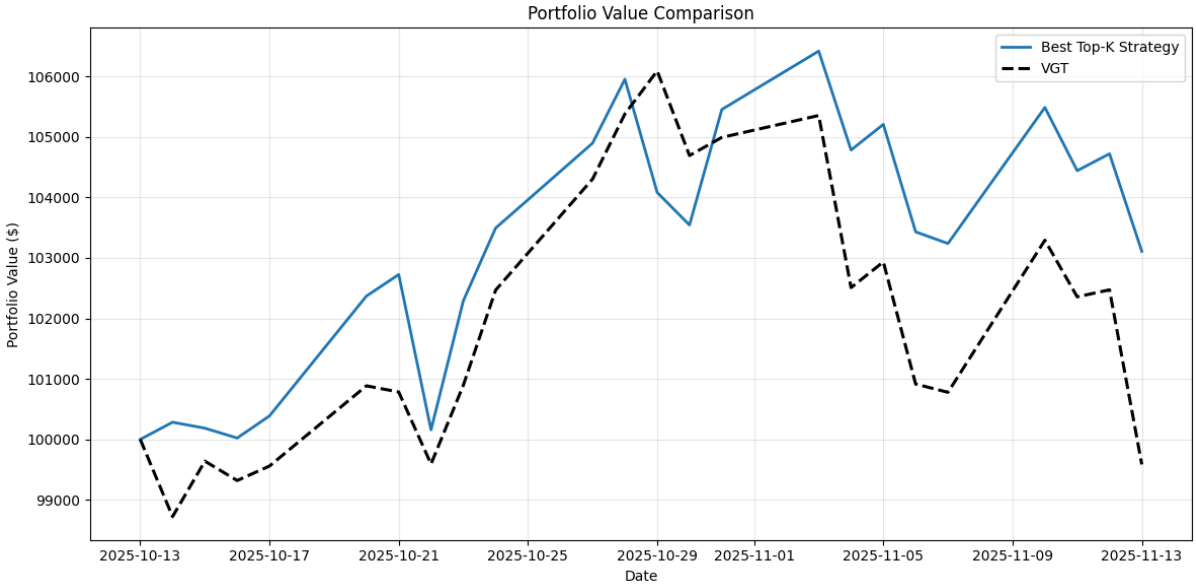

Figure 5: Performance for the best strategy compared to the VGT ETF on the test window.

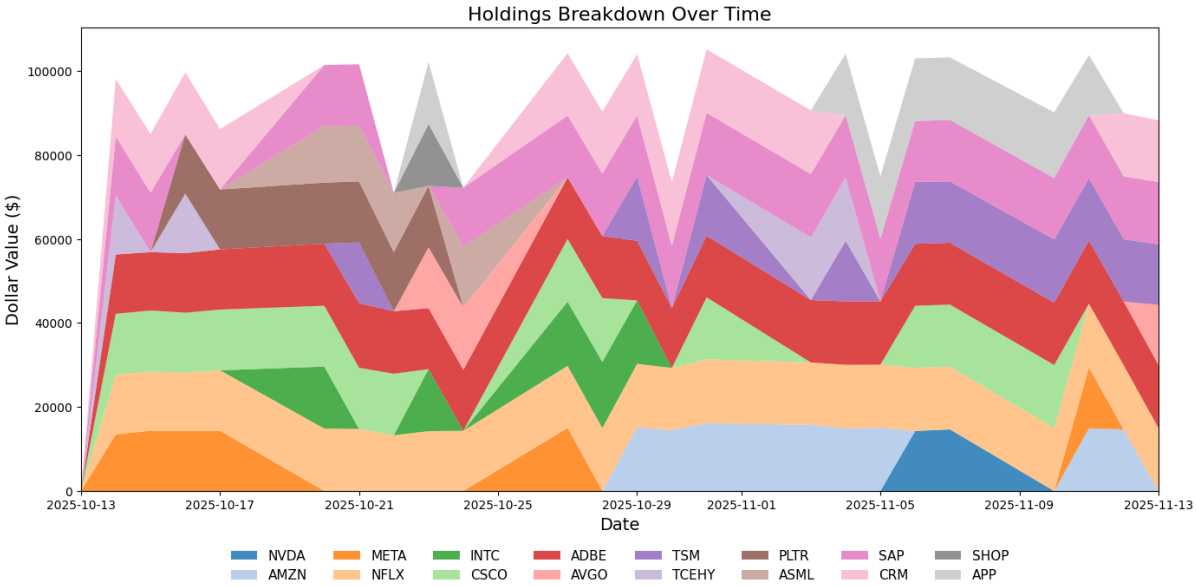

Figure 6: Stacked area chart of holdings over time.

5.2 Discussion

The results of the trading experiment show that the best-performing Top-K strategy, although profitable over the test window, only modestly outperforms the VGT benchmark. As seen in Figure 5, the strategy tracks the general upward trend of the market during this period, and the similarity of the two curves suggests that the model-driven allocations behave more like a momentum-following rule than a strongly predictive trading strategy.

Figure 7 provides additional insight into how the portfolio evolves. The holdings composition changes frequently, reflecting the daily rebalancing and sensitivity of the Top-K selection to small variations in predicted returns. Many stocks enter and exit the portfolio over short periods, indicating that the strategy responds aggressively to model-driven ranking changes. While this responsiveness enables occasional gains, it also introduces noise and instability in asset weights.

6 Conclusion and Future Work

This project set out to determine whether ARIMA, XGBoost, and Random Forest models, trained on ten years of daily price data and common technical indicators, could reliably forecast next-day stock prices and support profitable trading strategies. The evidence overwhelmingly indicates that they cannot. All three models exhibited low predictive power, with Mean Absolute Percentage Errors above $1 - 3\%$ and directional accuracy near 50%, comparable to random guessing. As a result, our Top-K trading strategy struggled to generate consistent returns.

Although hyperparameter tuning revealed configurations that performed well on the validation window, these gains did not persist into the test period, suggesting a lack of robustness and probable overfitting. The models tended to reproduce simple lag-following behavior rather than meaningful predictive structure. The backtesting results further reinforce this conclusion: while the best strategy briefly outperformed VGT, this advantage was neither stable nor large enough to be meaningful.

Future work should incorporate richer and more informative features, such as macroeconomic variables, volume data, sentiment indicators, or learned embeddings from deep models. Ultimately, successful short-term prediction in financial markets likely requires information beyond what was considered in this project.

References

[1] Samar K. Guharay, Gaurav S. Thakur, Fred J. Goodman, Scott L. Rosen, and Daniel Houser. Analysis of non-stationary dynamics in the financial system. *Economics Letters*, 121(3): 454–457, 2013. ISSN 0165-1765. doi: https://doi.org/10.1016/j.econlet.2013.09.026. URL `https://www.sciencedirect.com/science/article/pii/S0165176513004333`.

[2] Shakir Khan. Arima model for accurate time series stocks forecasting. *International Journal of Advanced Computer Science and Applications*, 2020.

[3] Abdullah Orhan and Necdet Saglam. Financial forecast in business and an application proposal: The case of random forest technique. *Journal of Accounting & Finance/Muhasebe ve Finansman Dergisi*, (99), 2023.

[4] Shubharthi Dey, Yash Kumar, Snehanshu Saha, and Suryoday Basak. Forecasting to classification: Predicting the direction of stock market price using xtreme gradient boosting. *PESIT South Campus*, 1:1–10, 2016.

[5] Sima Siami-Namini and Akbar Siami Namin. Forecasting economics and financial time series: Arima vs. lstm. *arXiv preprint arXiv:1803.06386*, 2018.

[6] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 785–794. ACM, August 2016. doi: 10.1145/2939672.2939785. URL `http://dx.doi.org/10.1145/2939672.2939785`.

[7] Felipe Barboza Oriani and Guilherme P. Coelho. Evaluating the impact of technical indicators on stock forecasting. In *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1–8, 2016. doi: 10.1109/SSCI.2016.7850017.

[8] Abdelatif Hafid, Maad Ebrahim, Mohamed Rahouti, and Diogo Oliveira. Cryptocurrency price forecasting using xgboost regressor and technical indicators. In *2024 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 1–6, 2024. doi: 10.1109/IPCCC59868.2024.10850357.

[9] Mark R Segal. Machine learning benchmarks and random forest regression. 2004.